Alternatives Considered

Risk Scoring Alternatives

Three alternatives to the deterministic weighted formula were considered for risk scoring: a supervised machine learning model, a pure rule-based threshold system without LLM involvement, and third-party student engagement analytics platforms. The deterministic formula was chosen over all three on the basis of explainability, data availability, and operational simplicity.

A supervised ML model (logistic regression, gradient boosting, or a neural network) would produce more accurate predictions if trained on historical outcome data linking student engagement patterns to actual re-engagement outcomes after facilitator intervention. That outcome data does not exist yet. Boon Academy has not systematically tracked which interventions worked and which did not. Building an ML model without outcome labels would produce a model that predicts current engagement patterns rather than future outcomes - which is exactly what the deterministic formula already does, without the opacity of a trained model. An ML approach should be revisited after six months of structured outcome tracking.

A rule-based system without LLM (e.g., "attendance below 60% -> HIGH risk, send this template message") would be faster and cheaper than the current approach but would produce identical messages for all students in the same risk tier. The LLM's ability to write contextually differentiated messages - even given the same metrics - is the primary value driver for facilitator adoption. Facilitators who receive personalised message drafts are more likely to send them without modification, which is what drives the intervention rate from 30% to 80%+. Removing the LLM removes this personalisation benefit.

Delivery Method Alternatives

Four delivery method alternatives were evaluated: n8n and Zapier (visual workflow automation), Airtable with automation features, the WhatsApp Business API for direct message sending, and the current approach of generating CSV drafts for manual sending. The current approach was retained because it is the only option that is fully offline-capable, version-controlled, and testable.

n8n and Zapier store workflow logic as visual node graphs that cannot be version-controlled in git, cannot be unit-tested, and cannot be debugged with standard developer tools. When a node fails, diagnosis requires navigating a web UI rather than reading a Python traceback. At the scale of 20 campuses with daily runs, the probability of a workflow failure in any given month is high enough that debuggability is a critical requirement - not a nice-to-have.

The WhatsApp Business API requires carrier approval (a process that takes weeks and has approval uncertainty), charges per message sent (typically $0.005 - $0.08 per message depending on country and template type), and requires a persistent server to handle delivery webhooks. For a pilot at 20 campuses sending approximately 120 messages per day, the approval process, ongoing cost, and server maintenance outweigh the benefit of automated sending, especially given that facilitator review before sending is a non-negotiable product

requirement.

Infrastructure Alternatives

Docker containerisation, a scheduled cloud job, and SQLite persistence were all evaluated and deferred rather than excluded permanently. Docker would provide environment reproducibility across facilitator machines but adds container orchestration knowledge requirements that are unrealistic for a one-person technical team at 20 campuses. The requirements.txt with pinned versions provides adequate reproducibility for the current deployment model.

A scheduled cloud job (GitHub Actions cron, AWS Lambda, or a Railway cron job) would eliminate the need for a facilitator to manually trigger the pipeline each morning. This is a worthwhile investment when the pipeline is running reliably on real data and the team has validated the output quality. It was deferred from the initial build because debugging a cloud job failure is significantly harder than debugging a local script failure - starting with a local script lowers the initial debugging burden and speeds up iteration during the validation period.

SQLite persistence would allow incremental runs (only process new data since the last run) rather than full re-ingestion of all CSVs daily. This is not needed at the current scale of 300 students where full ingestion takes under one second. The CSV -> SQLite migration path is documented in the scalability section and is the recommended next infrastructure step when campus count exceeds 50. No code change outside ingestion.py is required to execute that migration.

What Is Worth Building Next

Four improvements are worth building in the near term, in priority order: real student data hookup, risk weight calibration with the academic director, dialect selection per campus, and a scheduled daily trigger. These are not Phase 7-8 internal roadmap items - they are production readiness requirements that determine whether the pipeline produces genuinely useful output on real data.

Connecting the pipeline to real student data (replacing the synthetic CSVs) is the highest-priority next step. The synthetic data validates pipeline mechanics but cannot validate output quality or facilitator utility. Risk weight calibration should happen in parallel: the academic director should review the current weights (0.35 attendance, 0.30 practice, 0.20 trend, 0.15 notes) and adjust them based on their knowledge of which engagement signals actually predict student disengagement at Boon Academy. Until the weights are reviewed, the risk scores are informed guesses rather than validated predictions.

Dialect selection (Modern Standard Arabic vs. Gulf dialect per campus) would make the WhatsApp messages significantly more natural for recipients. Currently the model writes in whatever Arabic dialect it infers from the prompt context. Adding a campus-level dialect configuration and including it in the cohort prompt would improve message quality without any change to the pipeline architecture - it is a data configuration change. A scheduled daily trigger (operating-system task scheduler or a simple cron job) would remove the manual step of running the pipeline each morning, which is likely to be forgotten during busy periods, reducing the consistency of intervention coverage.